

PowerShell Malware Detection & Threat Hunting Report

Adversary Simulation, Endpoint Telemetry Analysis, and SOAR Automation

Host	The-Analyst	Date	May 28, 2026
Evaluated:		Generated:	
Investigator:	SOC Blue Team	Frameworks:	MITRE ATT&CK / OSSEM

1. Executive Summary

This comprehensive report outlines the engineering, validation, and automated triage of advanced PowerShell evasion tactics observed within our enterprise environment. Adversaries frequently leverage native system binaries like powershell.exe to bypass execution restrictions, download malicious payloads, and run obfuscated commands in memory, evading traditional endpoint security postures.

Through controlled simulation executed on the host The-Analyst, endpoint telemetry was aggregated utilizing Microsoft Sysmon and native Windows operational logging. Events were ingested via a SIEM platform (Splunk), parsed using structured behavioral analysis patterns, and subsequently routed to an automated Security Orchestration, Automation, and Response (SOAR) architecture powered by n8n and enriched via the VirusTotal API. This implementation creates a closed-loop defensive pipeline capable of reducing mean time to detect (MTTD) and mean time to respond (MTTR) down to seconds.

2. Adversary Simulation Framework (Red Team Execution)

To challenge behavioral detection models, multiple high-fidelity tactical test scenarios were executed to map cleanly against the MITRE ATT&CK Matrix. The following distinct attack techniques were simulated:

Technique 1: Execution Policy Bypass & Profile Evasion (MITRE T1059.001)

Attackers execute code natively without loading administrative restriction configurations or localized user scripts.

```
powershell.exe -ExecutionPolicy Bypass -NoProfile -Command "Write-Host 'execution policy bypass'"
```

Technique 2: Hidden Window Payloads & Malicious Ingestion (EICAR Stager)

Simulates an automated stager designed to operate without visual interaction, contacting remote infrastructure to ingest a secondary malicious implant into the \$env:TEMP directory.

```
powershell.exe -WindowStyle Hidden -NoProfile -Command "Invoke-WebRequest -Uri 'https://secure.eicar.org/eicar.com' -OutFile '$env:TEMP\eicar.com'"
```

Technique 3: Base64 Command Obfuscation (MITRE T1027)

Obfuscating code commands to bypass static string indexing engines and string matching layers on proxies or endpoints.

```
powershell.exe -ExecutionPolicy Bypass -EncodedCommand uwb0ageacg... (Decodes to: Start-Process calc.exe)
```

Technique 4: Anomalous Parent-Child Process Architectures

Web application runtimes, management planes, and document readers rarely require administrative orchestration shells. Execution was initiated through a non-standard parent to mimic an application server or macro exploitation scenario:

```
C:\Program Files  
odejs  
ode.exe -> spawning -> powershell.exe
```

3. Telemetry Ingestion & Log Deep-Dive

Standard Security Event Auditing often fails to supply the structural command context necessary to evaluate administrative context and parameters. Defense validation relies heavily on advanced auditing telemetry logs.

Sysmon Operational Telemetry (Event ID 1: Process Creation)

Sysmon monitoring captured precise process executions on the host *The-Analyst*. It recorded exactly which parent binary created the target process, the executing security identifiers, and full inline command structures.

Critical Insight: Cryptic Parent Tracking

By tracking the parent image metadata, our tracking caught a dangerous runtime pattern: *C:\Program Files odejs ode.exe* initiating an administrative command session. In an enterprise web landscape, this signal heavily points toward remote code execution (RCE) or server-side exploit fulfillment.

PowerShell ScriptBlock Logging (Event ID 4104)

While Base64 obfuscation successfully masks execution syntax inside Process Creation events (Sysmon EID 1), it completely fails against native Windows ScriptBlock Auditing. Event ID 4104 records the unrolled, raw text instructions as they parse directly within the PowerShell engine engine core.

During triage, Event ID 4104 recorded a *Verbose* entry capturing the completely de-obfuscated script text payload:

```
Creating Scriptblock text (1 of 1):  
Start-Process calc.exe  
ScriptBlock ID: 717693a1-0c9a-4e32-9f70-c754515bb535
```

4. SIEM Detection Engineering (Splunk Integration)

With log pipes routing into Splunk, targeted Search Processing Language (SPL) rules were developed to filter baseline operations and elevate real-time behavioral threats.

Rule A: Evasion Flag Identification

Identifies command parameters attempting parameter abbreviation or window suppression configurations.

```
index=* "sysmon" EventCode=1
| eval CommandLine=lower(CommandLine)
| search CommandLine="*-enc*" OR CommandLine="*-encodedcommand*" OR CommandLine="*-nop*"
OR CommandLine="*-nopprofile*" OR CommandLine="*hidden*"
| table _time, Computer, User, ParentImage, Image, CommandLine
```

Rule B: Anomalous Parent Hunting

Triggers when administrative shells originate from non-standard parent runtimes.

```
index=* "sysmon" EventCode=1
| eval ParentImage=lower(ParentImage), Image=lower(Image)
| search Image="*powershell.exe" AND (ParentImage="*node.exe" OR ParentImage="*cmd.exe" OR
ParentImage="*mshta.exe" OR ParentImage="*wscript.exe")
| table _time, Computer, ParentImage, Image, CommandLine
```

5. Theory Automation: SOAR & Threat Intelligence Enrichment

Manual analysts cannot match automated exploit speeds. To minimize latency, an automated response blueprint was structured using an **n8n SOAR Workflow Engine** integrated with **VirusTotal Core APIs**.

```
[Splunk Detection Trigger] | ▼ (SecOps JSON Payload via HTTP Webhook) [n8n Automation Engine] | |→ [Regex / JSON Parser: Extracts Target URLs, IPs & Hashes] | |→ [VirusTotal API Lookup Node: Submits Request Context] | ▼ [Response Routing: High-Risk Slack Alert / Automated EDR Endpoint Isolation]
```

Step-by-Step Automation Execution Pipeline:

Workflow Phase	Action Mechanics	Value Provided
1. Webhook Ingestion	Splunk real-time alert fires a POST request containing the event JSON array to n8n listener endpoint.	Instant handover from log platform to automation loop, removing human verification delay.
2. Extract Artifacts	Internal expression nodes filter out strings matching network locations (e.g., <code>https://secure.eicar.org</code>).	Isolates indicator data elements cleanly out of massive strings for specialized lookup.
3. Intel Enrichment	n8n issues an automated API request to VirusTotal endpoints checking the context reputation profile.	Enriches raw data streams with reputation context and global indicator analytics.
4. Alert Decision	Conditional logic branches positive malicious hits (detections > 0) straight to prioritized queues.	Provides immediate confirmation to the analyst, separating administrative accidents from actual threats.

Automation Scenario: The Node.exe Compromise

When `node.exe` executes an encoded command downloading code onto host `The-Analyst`, Splunk triggers an alert. Within 500ms, the n8n pipeline parses out the payload source, evaluates it via VirusTotal threat scores, and updates the SOC chat space with full risk assessment links. This completely eliminates manual triage friction.

6. Strategic Hardening Recommendations

Detecting malicious operations represents only half of a strong security program. True cyber resilience requires reducing systemic script attack surfaces:

- **Enforce Constrained Language Mode (CLM):** Restricts PowerShell functions by locking out direct access to underlying Win32 APIs and arbitrary COM objects.
- **Deploy AppLocker / WDAC policies:** Limit PowerShell script invocation execution exclusively to white-listed directories signed by organizational keys.

- **Continuous Enterprise Auditing:** Ensure Group Policy Objects (GPOs) explicitly require ScriptBlock Logging (ID 4104) and Module Logging (ID 4103) globally across all network tiers.